

Reviewer Pack — Minimal Hodge Index and Resolution Proof Width Inequality

Entry 70a5276cbba7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 20:09:56 UTC

Minimal Hodge Index and Resolution Proof Width Inequality

Entry ID: 70a5276cbba7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 20:09:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every k -CNF formula φ with n variables, the resolution proof width $w(\varphi)$ satisfies the inequality $\log_2(n^{k+1}) \leq w(\varphi) + H(\varphi)$, where $H(\varphi)$ is the minimal Hodge index of φ 's associated algebraic variety over the complex numbers.

Rationale (proposer's reasoning):

The Hodge index measures the complexity of an algebraic variety and is related to the geometric properties of the variety. By connecting this invariant with resolution proof width, we may reveal new insights into the structure of SAT instances and their hardness.

Taxonomy category: AlgebraicGeometry × ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1189b4cf97616555

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the inequality $\log_2(n^{k+1}) \leq w(\varphi) + H(\varphi)$ holds for all generated k -CNF formulas φ with n variables, where $w(\varphi)$ is the resolution proof width and $H(\varphi)$ is the minimal Hodge index. The conjecture is falsified if there exists any k -CNF formula φ such that $\log_2(n^{k+1}) > w(\varphi) + H(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Hodge Theory' AND 'resolution proof complexity' AND 'proof width inequality' - 'algebraic geometry' AND 'minimal Hodge index' AND 'CNF formula resolution' - 'complex algebraic variety' AND 'resolution proof width' AND 'logarithmic inequality'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters for k-CNF generation
    n = 10 # Number of variables
    m = 2 * n # Number of clauses (2 literals per clause)
    k = 3 # Clause length

    # Generate a random k-CNF formula  $\phi$  with n variables and m clauses
    phi = []
    for _ in range(m):
        clause = set(random.sample(range(1, n + 1), k))
        phi.append(clause)

    # Convert the k-CNF to a list of literals
    literals = [0] * (n + 1)
    for clause in phi:
        for literal in clause:
            literals[literal] += 1

    # Calculate the resolution proof width  $w(\phi)$ 
    def dp11(phi):
        if not phi:
            return 0
        unit_clauses = [c for c in phi if len(c) == 1]
        if not unit_clauses:
            return max(dp11([c - {l} for c in phi if l not in c]) for l in literals if literals[l] == 1)
        literal, _ = unit_clauses[0]
        new_phi = [c - {literal} for c in phi if literal not in c]
        return 1 + dp11(new_phi)
```

```

w_phi = dpll(phi)

# Calculate the minimal Hodge index  $H(\varphi)$  (simplified example)
H_phi = sum(literals[l] ** 2 for l in range(1, n + 1))

# Check the inequality  $\log_2(n^{(k+1)}) \leq w(\varphi) + H(\varphi)$ 
lhs = math.log2(n ** (k + 1))
rhs = w_phi + H_phi

conjecture_holds = lhs <= rhs
counterexample = "" if conjecture_holds else f"lhs={lhs}, rhs={rhs}"

return {
    "metric_name": "resolution_proof_width",
    "metric_value": w_phi,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    # Compute mean/std of metric_value
    total_metric_value = sum(r["metric_value"] for r in results)
    mean_metric_value = total_metric_value / len(results)
    variance = sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results)
    std_metric_value = math.sqrt(variance)

    # Compute fraction of seeds where conjecture_holds
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    # Determine the final result
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

) for l in literals if literals[l] > 0)

[illegible]

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify if the inequality $\log_2(n^{(k+1)}) \leq w(\varphi) + H(\varphi)$ holds for all generated k -CNF formulas. Therefore, the verdict is inconclusive. | next: Re-run the test without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22934
2	propose	ollama_remote	glm4:latest	0	0	14117
3	preregistration	ollama_remote	glm4:latest	0	0	10101
4	novelty	ollama_remote	glm4:latest	0	0	9125
5	novelty	ollama_remote	glm4:latest	0	0	15189
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16329
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10736
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13368
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9974
10	judge	ollama_remote	glm4:latest	0	0	15672

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137544 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/70a5276cbba7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/70a5276cbba7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/70a5276cbba7.tar.gz (if generated)